

Language Reference Manual for Mini-C

Project: CS301P Compiler Design Lab

Group ID: 17

Authors

Surya Ashish

Dhyanam

Rohan

Contents

1	Introduction	3
2	Program Structure	3
3	Lexical Elements	3
3.1	Keywords	3
3.2	Identifiers	4
3.3	Literals	4
3.4	Comments	5
4	Data Types and Declarations	5
4.1	Scalar Types	5
4.2	Variable Declarations	5
5	Expressions and Operators	5
5.1	Arithmetic Operators	5
5.2	Relational Operators	6
5.3	Assignment Operator	6
6	Statements and Control Flow	6
6.1	Conditional Statements	6
6.2	Iteration	6
6.3	Return Statement	7
7	Functions / Procedures	7
8	Semantic Rules and Error Messages	7
8.1	Scope Rules	7
8.2	Type Checking	7
8.3	Common Errors	7

9	Sample Programs	8
9.1	Factorial Implementation	8
10	Built-in Features / Limitations	8
10.1	Memory Management	8
10.2	Compiler Optimizations	8
10.3	Limitations	9
11	Conclusion	9

1. Introduction

Mini-C is a high-performance, statically typed programming language designed for structured programming and tensor-based operations.

This manual provides the formal specification of Mini-C syntax, data types, execution model, and programmer-visible behavior. It is intended for users who wish to write valid Mini-C programs compatible with the developed compiler.

2. Program Structure

A Mini-C program consists of:

- Global variable declarations
- Function definitions
- Statement blocks

Execution begins at the entry point defined by the `main()` function.

General structure:

```
int globalVar;  
  
int helper() {  
    return 1;  
}  
  
int main() {  
    return 0;  
}
```

3. Lexical Elements

3.1 Keywords

The following reserved keywords are supported:

- `int`

- if
- else
- while
- return
- alloc
- free

3.2 Identifiers

Identifiers must:

- Begin with a letter
- Be followed by letters or digits

Examples:

- count
- total1
- index2

3.3 Literals

Mini-C supports decimal integer literals.

Examples:

- 42
- 1024
- 0

3.4 Comments

Single-line comments:

```
// this is a comment
```

Block comments:

```
/* multiple  
   line  
   comment */
```

4. Data Types and Declarations

4.1 Scalar Types

The primary built-in type is:

- `int` : signed 64-bit integer

4.2 Variable Declarations

Variables must be declared before use.

Example:

```
int count = 0;  
int sum;
```

5. Expressions and Operators

Mini-C supports standard C-like precedence.

5.1 Arithmetic Operators

- `+` (addition)
- `-` (subtraction)
- `*` (multiplication)
- `/` (division)

- % (modulo)

5.2 Relational Operators

- ==
- !=
- i
- i
- i=
- i=

5.3 Assignment Operator

- =

Example:

```
x = a + b * c;
```

6. Statements and Control Flow

6.1 Conditional Statements

```
if (condition) {  
    ...  
} else {  
    ...  
}
```

The `else` block is optional.

6.2 Iteration

```
while (condition) {  
    ...  
}
```

The block executes while the condition is non-zero.

6.3 Return Statement

```
return expression;
```

This exits the current function and returns a value.

7. Functions / Procedures

Functions are defined using a return type, name, and parameter list.

Example:

```
int square(int x) {  
    return x * x;  
}
```

Mini-C follows the System V AMD64 ABI for parameter passing during compiled execution.

8. Semantic Rules and Error Messages

8.1 Scope Rules

Variables are valid only within the block `{ }` where they are declared.

Nested blocks support local scoping.

8.2 Type Checking

Arithmetic operations require integer operands.

Assignments must preserve type compatibility.

8.3 Common Errors

- Undeclared Variable: Using an identifier before declaration.
- Duplicate Declaration: Declaring a variable twice in same scope.
- Type Mismatch: Assigning incompatible values.
- Invalid Syntax: Missing semicolon or unmatched braces.

9. Sample Programs

9.1 Factorial Implementation

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

int main() {
    int result = factorial(5);
    return result;
}
```

10. Built-in Features / Limitations

10.1 Memory Management

Built-in functions:

- `alloc(size)`
- `free(ptr)`

These support dynamic heap allocation.

10.2 Compiler Optimizations

The compiler automatically performs:

- Constant Folding
- Dead Code Elimination
- Common Subexpression Elimination

10.3 Limitations

- No floating-point support
- No user-defined struct types
- Limited pointer arithmetic

11. Conclusion

Mini-C provides a robust framework for developing efficient procedural applications with emphasis on low-level memory control and optimized assembly generation.

It serves as an educational language demonstrating modern compiler implementation techniques.